

Lecture 3 - Quality Attributes: Case Study Presentations in Week 4

Topic: Quality Attributes, Scalability, Throughput, and Security

Format: Each Group: 5-10-Minute Live Presentation & Discussion

Case Study Presentations

Group 1: The "Morning Rush" Architects

Scenario: The Midland Hotel is overhauling its lobby system. The front desk staff are constantly stressed, especially during the 9:00 AM check-out rush and the 3:00 PM check-in wave. You have been hired to define the system's quality standards.

Presentation Prompts:

1. **QA Identification:** What are the specific Quality Requirements relevant to the *Check-In* and *Check-Out* tasks?

=> Specific Quality Requirements related to the Check-In and Check-Out tasks including:

- * Performance Efficiency (Time Consuming): According to group opinion, the system must remain minimal under intense concurrent load until 9:00 AM to 3:00 P.M

- * Availability/Reliability: The system must guarantee near-zero downtime (99.9% operational uptime). A crash during rush hours completely paralyzes the lobby.

- * Usability (Operability and Learnability): The user interface must be highly optimized for front-desk staff. It should support rapid keyboard shortcuts and clean data-entry forms to minimize human processing time per guest.

- * Security (Data Protection): Strict encryption and access control for guest data, such as passport/identity scans during Check-In, and credit card/billing transactions during Check-Out.

2. **Compare & Contrast:** What quality requirements are similar between these two tasks, and which ones are different? (Hint: Think about what the guest is doing/feeling during each).

=> Similarities:

- * High Performance Under Stress: Both of them require the software to handle sudden spikes in concurrent traffic without degrading speed. In both scenarios, guests are highly sensitive to long queues.

- * High Availability Needs: A failure in either task results in immediate operational chaos at the physical lobby desk.

=> Differences:

Feature	Check-in	Check-out
Guest Psychology	Guests wanna have check-in rooms immediately during long hours driving way to rest or prepare a crucial meeting.	Guests wanna check-out early due to late flights, trains, or morning business schedule => Time Constraints

Data Flow Focus	Data Ingestion & Filtering (Mixed Read/Write): Querying a Booking ID, verifying that the Room State is <i>Cleaned</i> and <i>Inspected</i> , and capturing identification data.	Heavy Financial Transaction (Heavy Write Path): Instantly consolidating all dynamic, multi-source <i>Incidental Expenses</i> into an immutable final invoice and settling payments.
Worst Case Risk	Frustrated guests, bad initial impressions, and poor customer satisfaction reviews.	Guests missing their flights. This introduces severe financial and legal liabilities for the hotel's brand reputation.

3. **Verifiable Spec:** Write one highly precise, verifiable requirement for the Check-Out task.

=> The system must be fast => Quality requirements must be measurable and verifiable. including:

- * **Measurable Condition:** Peak hours (8:00 AM - 10:00 AM) with 50 concurrent active workstations.

- * **Quantifiable Metric:** Less than 2.0 seconds

- * **Specific Action:** Consolidating expenses and rendering the invoice presentation.

4. **The Big Picture:** Taking a step back and looking at the *entire* Hotel Information System, what are the top 3 to 5 critical quality attributes?

=> **Reliability & Availability:** The hotel operates 24/7/365. The HIS must achieve 99.9% availability because mid-night check-ins, early check-outs, and late-night room service logging cannot stop.

=> **Scalability (Horizontal Elasticity):** The architecture must dynamically scale out (e.g., using CQRS read replicas) to handle massive seasonal holiday traffic surges without rewriting core code logic.

=> **Data Integrity & Consistency:** Total elimination of data anomalies like double-booking (assigning the same vacant room to different guests at the same time). This requires strict data transaction boundaries (ACID compliance) and database row-level locking.

=> **Security & Compliance:** The system handles massive volumes of sensitive personal identifiable information (PII) and global financial data. It must strictly comply with industry standards such as PCI-DSS for card payment safety and data privacy laws.

Group 2: The Infrastructure Engineers

Scenario: A major online booking engine, "GlobalStays," works perfectly for 11 months of the year. However, every Black Friday, the site completely crashes, losing millions of dollars in bookings. Management wants to know why the system can't handle the traffic.

Presentation Prompts:

1. **Throughput Analysis:** Explain to management what "Throughput" means. Why did the previous architects fail by designing the system based on "average throughput" instead of "peak throughput"?

=> What is Throughput:

- **Definition:** In system architecture, **Throughput** is the number of successful actions or transactions a system can process within a given timeframe.
- **Measurement:** It is typically measured in **Requests Per Second (RPS)** or **Queries Per Second (QPS)**.
- **The Analogy:** Think of throughput as the width of a highway. It dictates how many cars (user requests) can pass through a specific point every second.

=> Defining the Terms: What are Average and Peak Throughput:

- **Average Throughput:** This is the total volume of traffic the website receives over a long period (e.g., an entire year or 11 months), divided evenly down to the second. It represents our steady, everyday baseline traffic.
- **Peak Throughput:** This is the absolute maximum traffic intensity the system experiences during a sudden, concentrated surge. It represents the highest volume of concurrent requests hitting our servers at a single point in time.

=> Why the "Average" Approach Failed:

The fatal flaw? **Web traffic is never flat, smooth, or predictable.** It fluctuates violently based on real-world human behavior.

By designing for **average throughput**, the architects built a system prepared only for a quiet Tuesday afternoon. They completely ignored **peak throughput**—the absolute maximum traffic intensity during a sudden surge.

EX: When Black Friday midnight hits, traffic doesn't rise gradually; it hits the system like a tidal wave. It broke the system.

In conclusion, it fails because it can't handle overload in rush hours.

2. **The Scaling Solution:** To fix this, the system needs Scalability. Explain the difference between "Scaling Up" (Vertical) and "Scaling Out" (Horizontal).

=> The difference between "Scaling Up(Vertical)" and "Scaling Out(Horizontal)"

1. "Scaling Up" (Vertical Scaling)

- **The Concept:** Making a single, existing server bigger, faster, and more powerful.
- **The Action:** You keep the exact same machine or virtual instance, but you physically upgrade its hardware—adding more CPU cores, increasing the RAM, or swapping to faster Solid-State Drives (SSDs).
- **The Analogy:** If your single delivery van is overflowing with packages, you sell it and buy a massive, heavy-duty commercial cargo truck instead.

- **The Limitation:** You eventually hit a physical hardware ceiling. A single motherboard can only hold so much RAM and so many CPU cores. Once you buy the strongest machine on the market, you cannot scale any further.

2. "Scaling Out" (Horizontal Scaling)

- **The Concept:** Adding more servers to your existing pool of resources, rather than making a single server stronger.
- **The Action:** Instead of one giant machine, you connect multiple standard, smaller servers together. You place a **Load Balancer** in front of them to act as a traffic cop, distributing incoming user requests evenly across all available servers.
- **The Analogy:** Instead of relying on one massive cargo truck, you buy a fleet of 50 standard delivery vans that all work in parallel to distribute the load.
- **The Limitation:** There is virtually no hardware ceiling. If traffic doubles, you simply add 50 more standard servers to the pool.

=> Key point Scaling up has downtime problems while scaling out has more flexibility in exchange for more operating space.

3. **Recommendation:** Which scaling method would you recommend for a modern web application that experiences massive, unpredictable holiday spikes?

=> Horizontal Cloud Scaling (with Auto-Scaling):

Instead of trying to build or buy one single, massive server to handle the rush, this method works by adding or removing standard servers on demand using a cloud provider (like AWS or Google Cloud).

This approach ensures **absolute uptime** because there is no physical ceiling—the system just keeps adding servers to absorb the traffic wave. It is also incredibly **cost-efficient**: we only pay for the massive server capacity during the exact hours of the holiday rush, rather than paying for giant, overpowered hardware that sits idle for the other 11 months of the year.

Group 3: The Cybersecurity Consultants

Scenario: SecureBank recently updated its internal staff portal. To maximize security, they instituted 12-character rotating passwords, biometric scans, and a 3-minute inactivity timeout. Now, bank tellers are complaining that the system is completely unusable and goes offline constantly due to server load from encryption checks.

Presentation Prompts:

1. **AuthN vs AuthZ:** Clearly explain the difference between Authentication (AuthN) and Authorization (AuthZ) in this banking system.

=> **Authentication (AuthN) — "Who are you?"**

* **Definition:** Authentication is the process of validating and verifying the true identity of a user attempting to access the system. It proves that users are exactly who they claim to be.

* **In SecureBank's Context:** This is where the newly instituted security mechanisms reside. AuthN is executed when a bank teller presents their credentials at the shift start—specifically the **12-character rotating password** (something they know) and the **biometric scan** (something they are).

=> Authorization (AuthZ) — "What are you allowed to do?"

* **Definition:** Authorization is the process of verifying the specific permissions, access levels, and operational rights granted to an already authenticated identity.

* **In SecureBank's Context:** Once a teller is successfully authenticated via biometrics, AuthZ checks their role-based access control (RBAC) rules. For example, AuthZ determines whether an authenticated teller has the authority to approve a \$50,000 cash withdrawal, or if that specific command requires a branch manager's override signature.

2. **The Trade-off:** How does maximizing the quality attribute of **Security** directly negatively affect the quality attribute of **Availability**? Explain the mechanical/human reasons why this trade-off occurs.

=> Mechanical Reasons (System Load & Infrastructure Bottlenecks)

* **Cryptographic Overhead:** The system requires continuous encryption checks. Cryptographic hashing, token verification, and biometric pattern-matching are highly CPU-intensive operations.

* **Resource Exhaustion:** Because the system forces a strict **3-minute inactivity timeout**, hundreds of tellers across all branches are forced to constantly re-authenticate simultaneously throughout the day. This creates a massive, self-inflicted Distributed Denial of Service (DDoS) effect on the internal authentication servers.

* **Degraded Availability:** The server's CPU and memory pools become entirely exhausted by processing these cryptographic handshakes. As a result, the server drops legitimate business connections and goes offline, rendering the staff portal unavailable for actual banking transactions.

=> Human Reasons (User Behavior & Operational Stress)

* **Cognitive Overload:** Tellers handling high-stress client interactions cannot easily manage a aggressive 3-minute timeout alongside rotating 12-character passwords.

* **Security Fatigue:** When the software becomes unusable, humans naturally seek workarounds. Tellers may begin writing their rotating passwords on sticky notes under their keyboards or keeping dummy applications active to trick the 3-minute timeout.

* **Process Paralysis:** The constant disruption forces tellers to stop serving customers to repeatedly look into biometric scanners and re-type keys, artificially creating a massive queue at the bank counters and lowering the overall availability of banking services to the end client.

Group 4: The Tech-Debt Analysts

Scenario: A tech startup has a wildly popular mobile app. The users love it (it's fast and looks great). However, the developers are miserable; the codebase is a "spaghetti" mess, and adding a new feature takes three months.

Presentation Prompts:

1. **Internal vs. External:** Software qualities are categorized into Internal and External. Explain both categories. Based on the scenario, which category is succeeding and which is failing? Give examples.

=> External Quality (Visible to the User)

- * Definition: External qualities are the characteristics of the software that are directly experienced, observed, and felt by the end-users during runtime.

- * The Scenario Status: Succeeding.

- * Examples from the Text: The app is described as "wildly popular," "fast," and it "looks great." This means its external qualities—specifically Performance (speed) and Usability (aesthetic design and user experience)—are excellent.

=> Internal Quality (Visible to the Developers)

- * Definition: Internal qualities relate to the structural integrity of the source code, architecture, and documentation. These are completely invisible to the end-user but are heavily relied upon by engineers to maintain, extend, and test the software.

- * The Scenario Status: Failing.

- * Examples from the Text: The codebase is a "spaghetti mess," meaning it has high coupling, low cohesion, and zero architectural abstraction. This internal rot is forcing developers to be "miserable" because adding a single new feature takes an unacceptable three months

2. **General Attributes:** Give a quick summary of the 6 general software quality attributes (Availability, Modifiability, Performance, Security, Testability, Usability) and identify which specific attribute this startup is currently failing at.

=> **Availability:** The system's ability to remain operational, accessible, and fully functional to users during its scheduled uptime, resisting or recovering from system faults.

=> **Modifiability:** The ease with which a software system can be altered, extended, updated, or refactored to accommodate new business requirements, fix bugs, or improve performance.

=> **Performance:** The system's responsiveness, speed, throughput, and resource utilization when executing operations under specified benchmarked workloads.

=> **Security:** The system's capacity to protect sensitive data, resist unauthorized malicious attacks, and guarantee proper access control and identity validation.

=> **Testability:** The ease with which software components can be isolated, executed, and validated to demonstrate their correctness and uncover hidden defects.

=> **Usability:** The degree of ease, learnability, efficiency, and aesthetic satisfaction an end-user experiences when interacting with the application's interface.

